

# 如何使用 grcwa 进行优化

使用 grcwa 包必须在 Python 环境下，如果不了解 Python，请预先学习。比如

<https://www.runoob.com/python3/python3-tutorial.html>

grcwa 在的 github 上的仓库 <https://github.com/weiliangjinca/grcwa>

grcwa (autoGradable RCWA) 是老范组开发的一个可以用于仿真、优化光子器件的 Python 包。RCWA (严格耦合波分析) 相比于 FDTD 这种时域上计算的仿真，每次只能计算一个波长下的结果，所以如果想要得到光谱信息，需要多次调用计算；但是另一方面，RCWA 不存在收敛与否的问题，只有级次和网格数目多少带来的计算精度与时间问题。通常来讲，使用 grcwa 进行优化基于梯度下降算法。当然，单纯将它当成一个电磁求解器也可以。

梯度下降是普遍适用的优化算法，它需要用到另外两个包，autograd 和 nlopt。autograd 可以帮你自动求解某一个函数对于自变量的梯度。nlopt (non-linear optimization) 是一个集成了很多非线性优化的工具，只要你把梯度信息给它，就可以直接使用里面的梯度下降。

从一个例子出发，可以快速上手 grcwa 的使用方法。样例中所设计的是一个三层的超表面结构：

- 最下层为金属反射衬底，上面两层为介质，最上层有周期性的超表面结构图案。
- 每一个周期内的图案为中间挖掉一个圆的矩形。
- 优化参数为：周期大小，介质层的厚度，矩形的长宽，圆的半径。
- 优化目标为：10 ~ 11 $\mu\text{m}$  波段内，反射率尽可能接近 0.5

## 1. 在程序的开头，有一些需要用到的包：

```
import sys
import time
import nlopt
import pandas as pd
import autograd
import autograd.numpy as np
import grcwa
grcwa.set_backend('autograd') # important!!
import matplotlib.pyplot as plt
```

- 如果某一个包没有，比如 nlopt，使用 pip install nlopt 命令下载安装。grcwa 包也可以直接用 pip 安装。

## 2. grcwa 仿真过程中的一些基本参数设置。它们通常来讲都不需要在仿真或者优化过程中被改动，可以设置成全局的：

```
# 假设这里出现的长度单位皆为 um
Nx, Ny = 100, 100
nG = 51
plane_wv = {'p_amp': 1 / np.sqrt(2), 's_amp': 1 / np.sqrt(2), 'p_phase': 0, 's_phase': 0}
```

- Nx 和 Ny 是一个周期内的横纵方向网格划分数目。和 FDTD 一样，RCWA 同样需要对模型划分网格，每一个格子内部认为介电常数是均一的。网格数划分多少跟结构的复杂程度有关。
- nG 为衍射级次的数目。这是 RCWA 计算中重要的参数，它直接影响到仿真结果的精确性和运行时间，选择一个合适的 nG 值非常重要。特别的，如果你只是想计算多层膜的光学特性，grcwa 也是完全可以胜任的，此时它与 TMM 本质上没有区别，nG 可以设置为 1，计算速度相当快。如果是复杂的二维超表面结构，nG 则需要设置的大一些，实际使用中可以调整 nG 大小观察结果有无显著区别来判断。
- plane\_wv 是光源的偏振参数。需要注意的是，最好令振幅为 1，这样仿真得到的结果直接就是反射率。如果振幅为 2，那程序返回的“反射率”可能会大于 1。
- 需要注意的是，grcwa 并没有明确规定使用的长度单位。这是因为在介电常数不变的情况下，对结构、波长这些物理模型整体缩放并不会改变最终的结果。所以使用者只需要自己约定在同一个程序中使用同一个单位即可。

## 3. 材料的介电常数设置：

```

e_vacuum = 1 + 0j

nk_0 = pd.read_csv('materials\\Si_0.25_14.txt', sep = ' ').to_numpy()
nk_1 = pd.read_csv('materials\\SiO2_7_50.txt', sep = '\\t').to_numpy()
nk_2 = pd.read_csv('materials\\Au_0.3_24.93.txt', sep = ' ').to_numpy()
print(nk_0.shape, nk_1.shape, nk_2.shape)

lambda_x = np.linspace(10, 11, 100)
e_0 = (np.interp(lambda_x, nk_0[:, 0], nk_0[:, 1]) + 1j * np.interp(lambda_x, nk_0[:, 0], nk_0[:, 2])) ** 2
e_1 = (np.interp(lambda_x, nk_1[:, 0], nk_1[:, 1]) + 1j * np.interp(lambda_x, nk_1[:, 0], nk_1[:, 2])) ** 2
e_2 = (np.interp(lambda_x, nk_2[:, 0], nk_2[:, 1]) + 1j * np.interp(lambda_x, nk_2[:, 0], nk_2[:, 2])) ** 2

def eps_at(lam):
    e0 = np.interp(lam, lambda_x, e_0)
    e1 = np.interp(lam, lambda_x, e_1)
    e2 = np.interp(lam, lambda_x, e_2)
    return [e0, e1, e2]

```

- 一般来讲，我们需要把可能用到的材料的介电常数信息预先导入进来，并设置好目标的波长范围。确保数据导入是正确的，`nk_0` 应该是一个三列的 `np.array`，这可以通过观察 `nk_0.shape` 来得到。
- 需要特别注意的是，与 FDTD 中直接设置材料的复数折射率不同，`grcwa` 中需要的是**介电常数**。这意味着如果你是从折射率网站下载的  $n + ik$  数据，需要先平方得到介电常数数据。
- 原始介电常数的数据点通常不是你需要计算的波长，使用 `np.interp` 线性插值来得到任意波长的介电常数。

#### 4. 归一化的网格坐标点：

```

x_lin = np.linspace(0, 1, Nx)
y_lin = np.linspace(0, 1, Ny)
ptx, pty = np.meshgrid(x_lin, y_lin, indexing = 'ij')
ptx, pty = ptx.flatten(), pty.flatten()

```

- 正如之前所言，`grcwa` 仿真也需要进行网格划分。这里我们要按照  $N_x$  和  $N_y$  的大小设置归一化的网格坐标点。`np.meshgrid` 计算横纵坐标序列的笛卡尔积得到所有  $N_x \times N_y$  个网格坐标点，这些网格点的横纵坐标分别于 `ptx` 和 `pty` 两个序列中。
- 这些坐标点并不会输入给 `grcwa` 包，它们只是出于方便，辅助你给网格中的每一个格点分配介电常数的。我们很快就会看到其作用。

#### 5. 构建图案：

```

def pattern(a, b, r):
    # 矩形使用相对切比雪夫距离
    fill1 = np.maximum(np.abs(ptx - 0.5) / 0.5 / max(a, 1e-7), np.abs(pty - 0.5) / 0.5 / max(b, 1e-7))
    fill1 = 1 / (1 + np.exp(233 * (np.minimum(2, fill1) - 1)))
    # 圆形使用相对欧几里得距离
    fill2 = np.sqrt((ptx - 0.5) ** 2 + (pty - 0.5) ** 2) / 0.5 / max(r, 1e-7)
    fill2 = 1 / (1 + np.exp(-233 * (np.minimum(2, fill2) - 1)))
    return np.minimum(fill1, fill2)

```

- 通常膜层的参数设置是非常简单的，而顶层带有图案的超表面则要复杂不少。在 `grcwa` 的建模逻辑中，一个光子器件需要自上而下被分成若干层，每一层要么是均一的膜，要么是带有特定介电常数分布的超表面结构。对于膜，需要传入厚度与介电常数两个参数；对于超表面，需要传入厚度，以及一个  $N_x \times N_y$  大小的序列，描述一个周期内每个格点上的介电常数。所以我们需要把诸如矩形长宽、圆半径等结构参数，转化为介电常数分布。
- 由于我们是在  $[0, 1] \times [0, 1]$  归一化的网格上，所以不需要考虑周期大小等问题。在这里我们约定  $a, b, r$  分别代表了矩形长宽、圆半径在周期内的相对比例。比如，当  $a = 1$  时，矩形长将占满整个周期， $a = 0.5$  时，矩形长占整个周期的一半；当  $r = 1$  时，圆恰好内切与整个周期。这些约定可以按照使用者自己的习惯自由更改，保证逻辑自洽即可。
- 假设介质和真空的介电常数分别为  $\epsilon_{meta}, \epsilon_{vacuum}$ ，格点的介电常数可以表示为  $e = \lambda \cdot \epsilon_{meta} + (1 - \lambda) \cdot \epsilon_{vacuum}$ ，`pattern` 返回值就是  $\lambda$  的分布。

- 一个直接的想法是，如果一个格点在矩形内且在圆外，那它的  $\lambda$  就是 1，否则就是 0。遗憾的是，这么做会导致  $\lambda$  分布与结构参数  $a, b, r$  之间为阶梯函数，间接导致最终的目标函数与自变量之间为阶梯函数。而阶梯函数的不连续性以及梯度处处为零，会让梯度下降算法失效。一个常用的技巧是，用激活函数来模拟或者说替代这个阶梯。  

$$f(x) = \frac{1}{1+e^{a(x-b)}}$$
可以模拟一个 0 与 1 之间的阶梯，其中  $a$  的大小用于调节阶梯的锐度， $a$  的正负用于调节阶梯的方向， $b$  用于调节阶梯的位置；这样梯度就可以保证存在了（这里有一个小细节， $a$  的大小不必取太大，可能会存在爆精度问题，通常两三百就够用）。现在我们只需要对每一个格点计算一个值，使得在图案边界上的格点值恰好为  $b$ ，边界内外的格点值分别在  $b$  的两侧。对于圆而言，直接使用相对欧几里得距离即可；对于矩形而言，使用相对切比雪夫距离。
- 只有矩形内且在圆外的格点， $\lambda$  才能为 1，使用 `np.minimum` 取交集。有需要，使用 `np.maximum` 取并集。

## 6. 调用 grcwa 计算结果：

```
def rcwa_calc(lam, theta, para):
    pd, t0, t1, a, b, r = para[0], para[1], para[2], para[3], para[4], para[5]
    epsilon = eps_at(lam)
    ep = e_vacuum + pattern(a, b, r) * (epsilon[0] - e_vacuum)

    L1, L2 = [1, 0], [0, 1]
    freq, phi = 1 / lam * (1 + 1j / 2 / 1e9), 0
    obj = grcwa.obj(nG, L1, L2, freq, theta, phi, verbose = 0)
    obj.Add_LayerUniform(100, e_vacuum)
    obj.Add_LayerGrid(t0, Nx, Ny)
    obj.Add_LayerUniform(t1, epsilon[1])
    obj.Add_LayerUniform(1, epsilon[2]) # 金属反射背板
    obj.Add_LayerUniform(100, e_vacuum)
    obj.Init_Setup(Pscale = pd)
    obj.MakeExcitationPlanewave(plane_wv['p_amp'], plane_wv['p_phase'], plane_wv['s_amp'],
    plane_wv['s_phase'], order = 0)
    obj.GridLayer_geteps(ep)
    R, T = obj.RT_Solve(normalize = 1)
    return np.abs(R)
```

- `lam`, `theta`, `para` 分别为入射波长、角度以及结构参数。`rcwa_calc` 返回计算的反射率。
- 首先需要根据波长得到对应的介电常数，对顶层的超表面构建图案，得到顶层  $N_x \times N_y$  的介电常数分布序列。
- `L1`, `L2` 是周期的基向量，一般来讲使用正交的单位向量 `[1, 0]`, `[0, 1]` 就可以了，这两个向量必须是常量，所以如果特别的，周期不是一个正方形单元，它的长宽比也只能是固定的。
- 光频率 `freq` 直接取 `lam` 的倒数，相位 `phi` 一般只需要取 0。
- `grcwa.obj` 将构造一个仿真的对象。
- 使用 `obj.Add_LayerUniform` 或者 `obj.Add_LayerGrid` 从上到下依次创建每一层。均匀膜层传入厚度与介电常数参数；格点层先传入厚度，介电常数参数等会儿再统一传入。需要注意的是，最上和最下必须要用真空层，不然仿真结果就会出错，真空层的厚度无所谓。
- `obj.Init_Setup` 进行初始化，利用 `Pscale` 设置周期长度，令它取值为 `pd`。
- `obj.MakeExcitationPlanewave` 设置入射波参数。
- `obj.GridLayer_geteps` 传入格点的介电常数分布。特别说明，这个例子中只有一层有超表面图案，所以传入的 `ep` 是一个长度为  $N_x \times N_y$  的序列，如果有多层比如  $k$  层超表面图案，那么 `ep` 应该是一个长度为  $k \times N_x \times N_y$  的序列，由自上而下每一层的介电常数分布拼接而成。
- 最后，只需要调用 `obj.RT_Solve` 即可得到计算结果。如果介电常数是复数，返回的 `R, T` 也是复数类型，取实部或者模长都可以。

## 7. 光谱计算：

```
def get_spec(theta, para):
    spec = np.zeros(lambda_x.size)
    for i in range(0, lambda_x.size):
        spec[i] = rcwa_calc(lambda_x[i], theta, para)
    return spec
```

- 开头讲到，由于 RCWA 每次只能计算单波长下的结果，所以得多次调用才能得到光谱的信息。

## 8. 优化的目标函数：

```
def func_target(para):
    theta, max_R = 0, 0
    for lam in lambda_x:
        max_R = max(max_R, np.abs(rcwa_calc(lam, theta, para) - 0.5))
    return max_R
```

- 上面的部分讲的是如何使用 grcwa 进行仿真。既然需要进行优化，那必须确定目标函数。同样的，需要注意避免在优化函数中出现阶梯函数之类的不连续、梯度为零的情况。
- 例子中的目标是让反射率尽可能接近 0.5，目标函数是让偏离 0.5 的最大值最小。

## 9. 优化函数：

```
def func_nlopt(x, grad_x):
    global counter
    grad_x[:] = grad_target(x)
    y = func_target(x)
    print('Step =', counter, ', para =', x, ', R =', y)
    counter += 1
    return y
```

- 这个 func\_nlopt 优化函数是传递给 nlopt 包的，计算当前自变量  $x$  的梯度值  $\text{grad}_x$ ，一般为固定写法，无需改动。

## 10. 优化流程的设置：

```
lower_bd = np.array([1, 1, 1, 0, 0, 0])
upper_bd = np.array([20, 10, 10, 1, 1, 0.8])
opt = nlopt.opt(nlopt.LD_MMA, 6)
opt.set_lower_bounds(lower_bd)
opt.set_upper_bounds(upper_bd)
opt.set_xtol_rel(1e-5)
opt.set_maxeval(100)
opt.set_min_objective(func_nlopt)
```

- 例子中需要优化的变量有 6 个，分别为  $pd, t_0, t_1, a, b, r$ 。lower\_bd 和 upper\_bd 是这些变量的上下界，它们会保证在梯度下降的过程中，变量不会超过这个范围。
- nlopt.opt 构造一个优化对象，需要传入变量个数。
- opt.set\_lower\_bounds 和 opt.set\_upper\_bounds 分别设置变量的上下界。
- opt.set\_xtol\_rel 设置优化的终止条件之一，表示当自变量的相对变化量小于  $10^{-5}$  时，结束优化。当然还存在诸如自变量的绝对变化量小于多少时结束的其他终止条件，不过相对变化通常更泛用。
- opt.set\_maxeval 设置优化的终止条件之二，表示迭代次数达到 100 次之后，结束优化。它的存在保证优化流程有着可预测的运行时间。
- opt.set\_min\_objective 最小化目标函数。如果想要最大化目标函数，可以使用 opt.set\_max\_objective 或者将目标函数取相反数。

```
init_x = np.array([12, 3, 10, 0.9, 0.6, 0.2])
best_x = opt.optimize(init_x)
```

- 最后，只需要传入初始状态 init\_x，调用 opt.optimize 即可得到优化后的状态。
- 初始状态对于最终优化结果的影响非常大。选择一个好的初始状态很重要。当然，可以选择和例程中一样多次随机初始状态，不过是否有效就说不准了。